

Threat Analysis Report

Lab 1: DevSecOps Fundamentals & SDLC Security Integration

1. Scope

This report documents a two-phase approach to secure software development, applied to a user data storage script. Phase 1 establishes an intentionally insecure baseline to demonstrate real security weaknesses. Phase 2 applies secure coding principles, NIST SSDF practices, and DevSecOps tooling to remediate each weakness, verify the fix, and detect tampering.

2. Two-Phase Approach

Phase 1 — Insecure Baseline (insecure_version.py)

A deliberately vulnerable script was built first, containing seven intentional flaws: a hardcoded encryption key, reversible XOR "encryption" instead of real encryption, no input validation, no error handling (crashes if users.json is missing), no integrity hashing (tampering is invisible), duplicated file-reading logic (violates DRY), and passwords printed directly to the console. Running this version and screenshotting the output shows these weaknesses are real and exploitable, not merely theoretical.

Phase 2 — Secure Remediation (secure_version.py)

Each flaw from Phase 1 was corrected individually: the key moved out of source code, real encryption (Fernet/AES) applied to passwords, input validation added, error handling added for missing/corrupt files, SHA-256 integrity hashing added per record, duplicated logic consolidated into shared functions, and console output masked. A persistent audit log (security_audit.log) was also added so every integrity check — pass or fail — is timestamped and recorded, standing in for what a SIEM tool such as Splunk would monitor in production.

3. Identified Risks and Mitigations

Risk	Mitigation	Likelihood / Impact
Encryption key stored in plain text / hardcoded in source code.	Store the key in its own file outside version control, or a secrets manager (e.g., AWS KMS, HashiCorp Vault). Rotate periodically.	Medium likelihood / High impact — if exposed, all passwords can be decrypted.
Insufficient input validation when adding new users.	Enforce minimum password length, validate email format, reject empty or duplicate usernames before writing to storage.	High likelihood / Medium impact — weak or malformed data accepted silently.
File tampering: users.json / secure_users.json can be edited directly on disk, bypassing the application.	SHA-256 integrity hashing per record (Task 4), combined with a persistent audit log reviewed via log analysis (Splunk).	Medium likelihood / High impact — demonstrated directly using users_tampered.json.
No authentication or access control around	Wrap storage operations behind an authenticated service/API with role-based	Medium likelihood / High impact — relevant once this moves beyond a lab

the script itself.	access control rather than direct file access.	exercise.
Plain SHA-256 hash offers integrity but not tamper-proof authenticity (attacker controlling both data and hash could forge a match).	Use an HMAC (keyed hash) or digitally sign records so only a holder of the secret can produce a valid hash.	Low likelihood in this lab context / Medium impact — a known design limitation.

4. NIST SSDF (SP 800-218) Practice Mapping

Each Phase 2 fix was mapped to a specific NIST Secure Software Development Framework practice, to ground the remediation in a recognized standard rather than ad hoc judgment.

Phase 2 Fix	SSDF Practice	Description
Threat modeling before coding (OWASP Threat Dragon)	PW.1	Define security requirements and design to meet them, before implementation begins.
Encrypted storage, SHA-256 integrity hashing	PS.1	Protect all forms of code and data from unauthorized access and tampering.
secure_version.py replacing insecure_version.py	PW.4	Create source code that adheres to secure coding practices.
Side-by-side comparison of insecure vs. secure code	PW.7	Review and analyze code to identify vulnerabilities before release.
security_audit.log + Splunk log analysis	RV.1	Identify and confirm vulnerabilities / incidents on an ongoing basis.
This threat analysis report	RV.2	Assess, prioritize, and remediate identified vulnerabilities.

5. Manual Dynamic Testing Findings

Because this application is a local command-line script rather than a networked web service, a formal DAST tool (e.g., OWASP ZAP) cannot scan it directly — DAST tools attack a running web application over HTTP. As a substitute, a manual dynamic testing pass (dynamic_test.py) fed 12 edge-case and hostile inputs into the running program's validate_input() function and recorded the actual runtime behavior, rather than reading the source code.

Result: all 12 test cases behaved as expected. Two genuine gaps were surfaced:

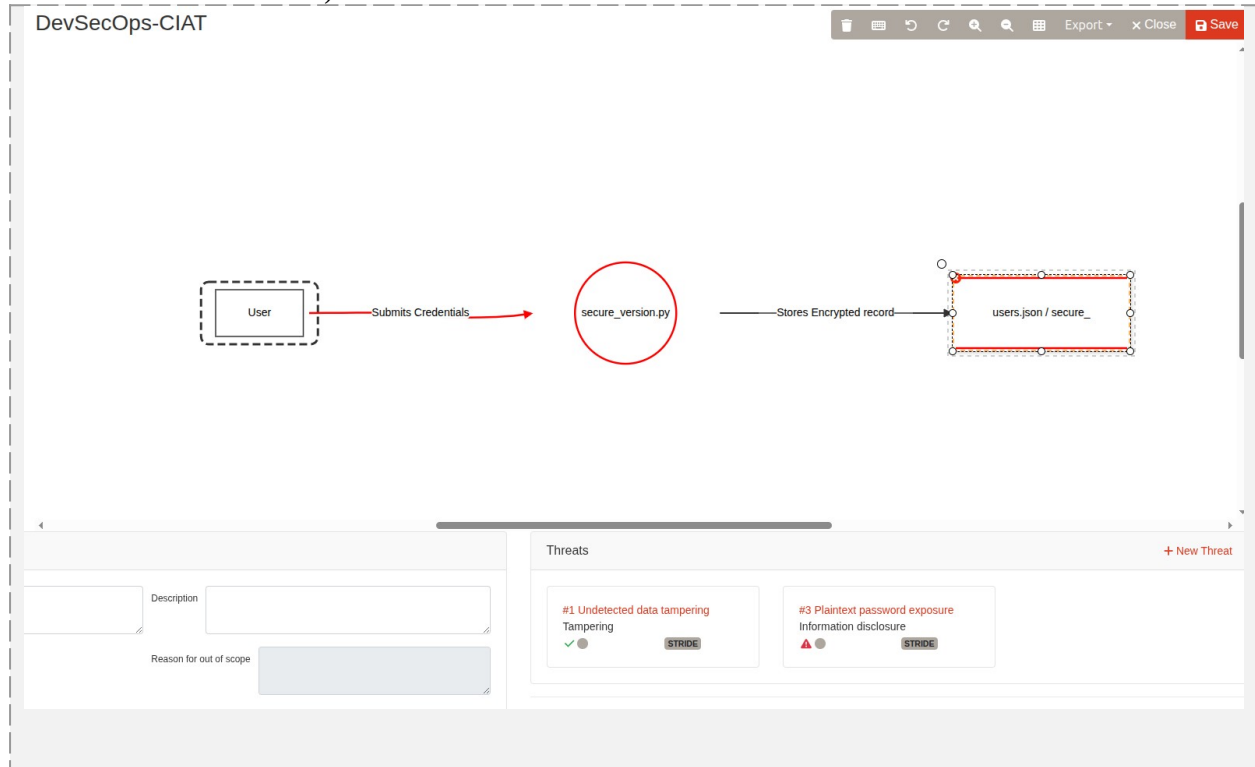
1. Whitespace-only passwords are accepted by the length check, since whitespace characters count toward the 8-character minimum. This is a real, unaddressed gap.
2. Special characters (script-injection-style strings, SQL-injection-style strings) are accepted without sanitization. This is inert in the current design, since the application stores data in JSON files and never constructs a SQL query or renders HTML — but the same input would be a serious risk if this application were later connected to a SQL database or a web front end.

6. Real-World Tooling Evidence

To connect this lab to actual DevSecOps tooling (rather than only Python scripts), three tools referenced in the accompanying CIA Triad discussion post were applied directly to this project.

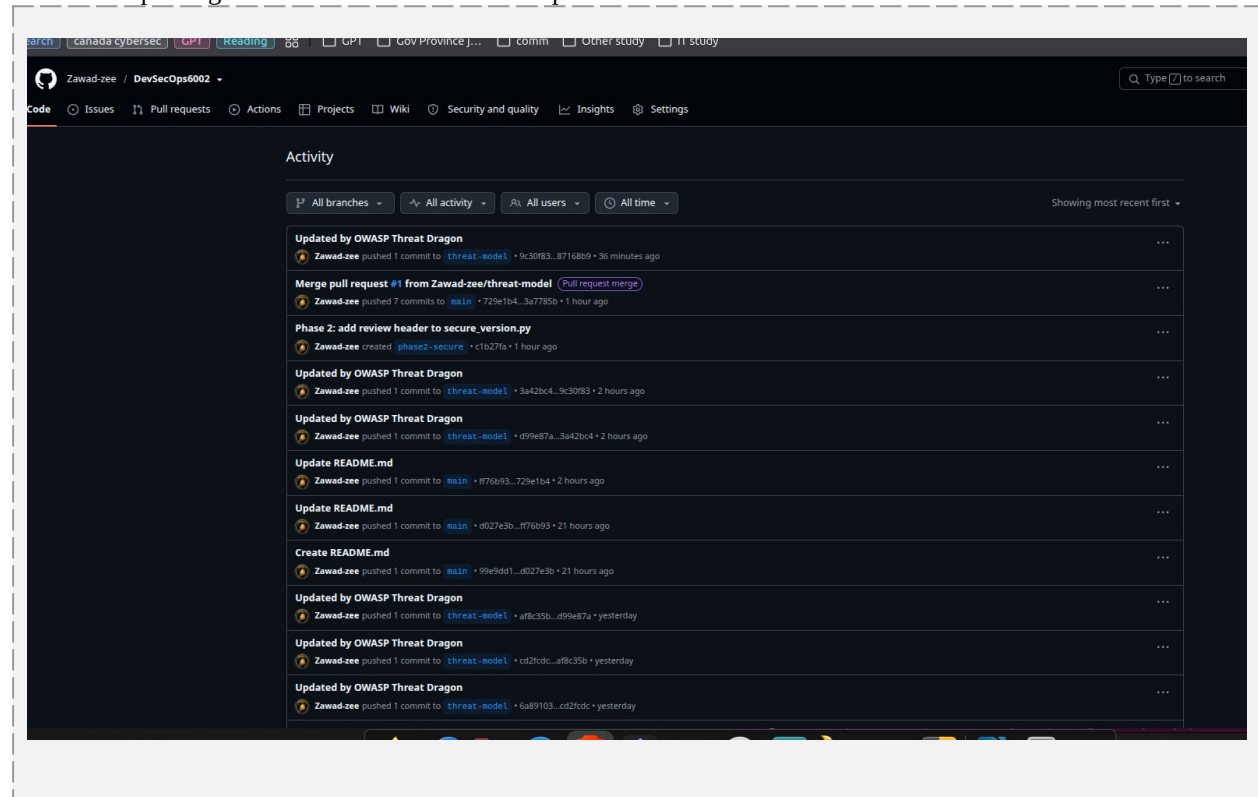
6.1 OWASP Threat Dragon — Planning Phase

A threat model was built for this application prior to remediation, diagramming the User, the script process, and the data store, with a trust boundary between them. Each Phase 1 flaw was logged as a formal STRIDE threat (e.g., hardcoded key and missing integrity hash both categorized under Tampering; console-printed passwords categorized under Information Disclosure).



6.2 GitHub — Development and Release Phases

The Phase 1 baseline was committed to a Git repository and pushed to GitHub. Phase 2 remediation was developed on a separate branch (phase2-secure) and merged via Pull Request, demonstrating the peer-review checkpoint from the Development phase. A tagged Release (v1.0-secure) marks the approved, remediated version, corresponding to the Release phase gate described in the discussion post.



Conversation 0 Commits 6 Checks 0 Files changed 1



Zawad-zee commented 1 minute ago

Owner ...

Phase 2 secure implementation, reviewed and ready to merge



Zawad-zee added 6 commits yesterday

Created by OWASP Threat Dragon

6a89103

Updated by OWASP Threat Dragon

cd2fcdc

Updated by OWASP Threat Dragon

af8c35b

Updated by OWASP Threat Dragon

d99e87a

Updated by OWASP Threat Dragon

3a42bc4

Updated by OWASP Threat Dragon

9c30f83

Zawad-zee merged commit 3a7785b into main now

Revert



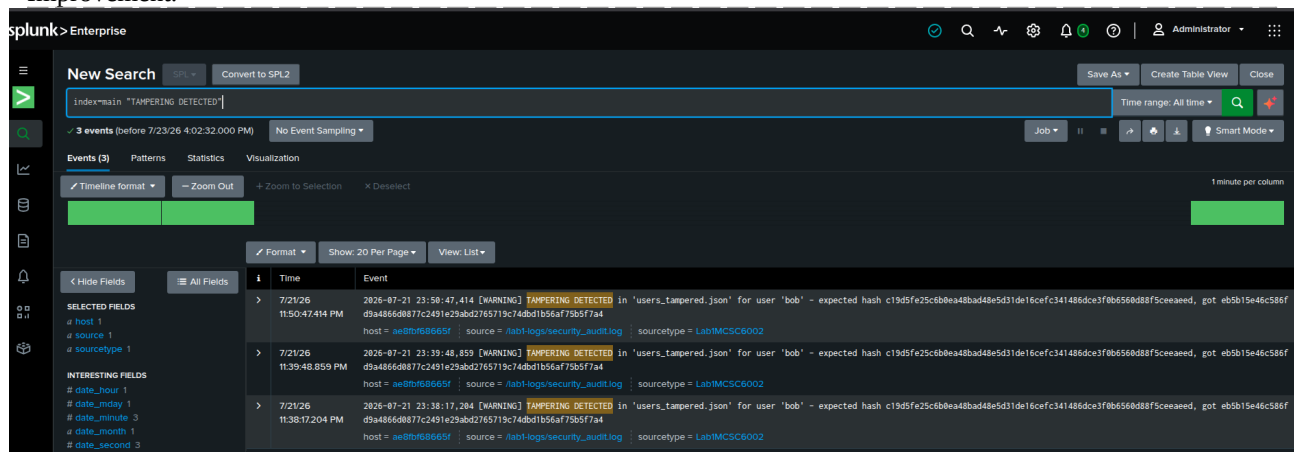
Pull request successfully merged and closed

You're all set — the threat-model branch can be safely deleted.

Delete branch

6.3 Splunk — Operations and Continuous Monitoring Phases

security_audit.log was ingested into a locally run Splunk instance (Docker) as a monitored file input. A search query ("TAMPERING DETECTED") successfully surfaced the log entry generated when secure_version.py checked users_tampered.json against the original reference hashes, correctly flagging the modified bob record. This demonstrates the same tool serving two distinct purposes described in the discussion post: real-time detection during Operations, and the same searchable log data supporting longer-term trend review during Continuous Monitoring & Improvement.



7. Summary

The two-phase approach demonstrates the core DevSecOps argument in practice: security is not a single late-stage gate, but a set of principles applied continuously and verified with real tooling. Phase 1 shows what happens without those principles; Phase 2 shows the same application hardened against confidentiality, integrity, and availability threats, backed by a recognized standard (NIST SSDF), a documented threat model (Threat Dragon), a version-controlled review process (GitHub), and real log-based detection (Splunk).